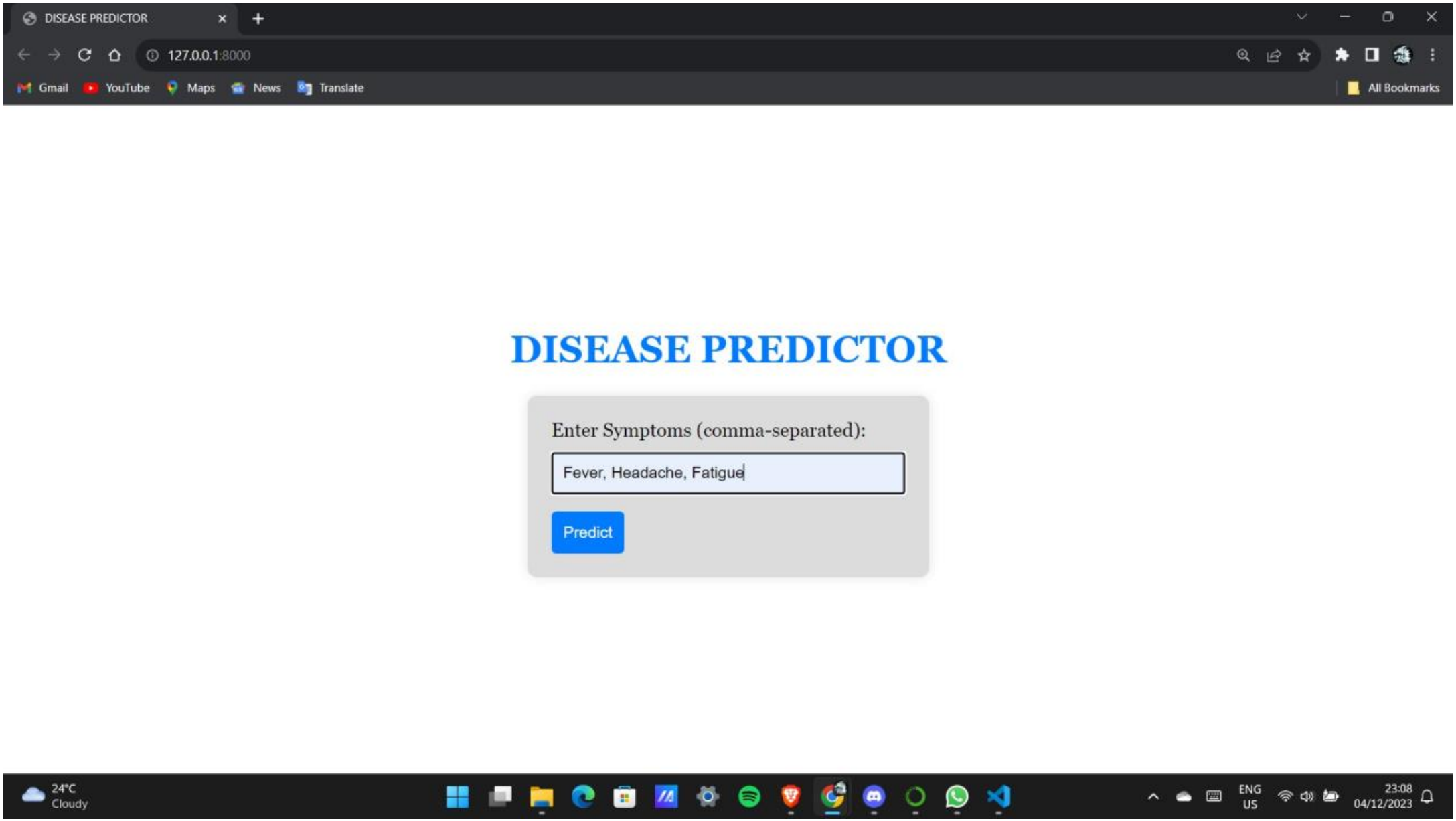
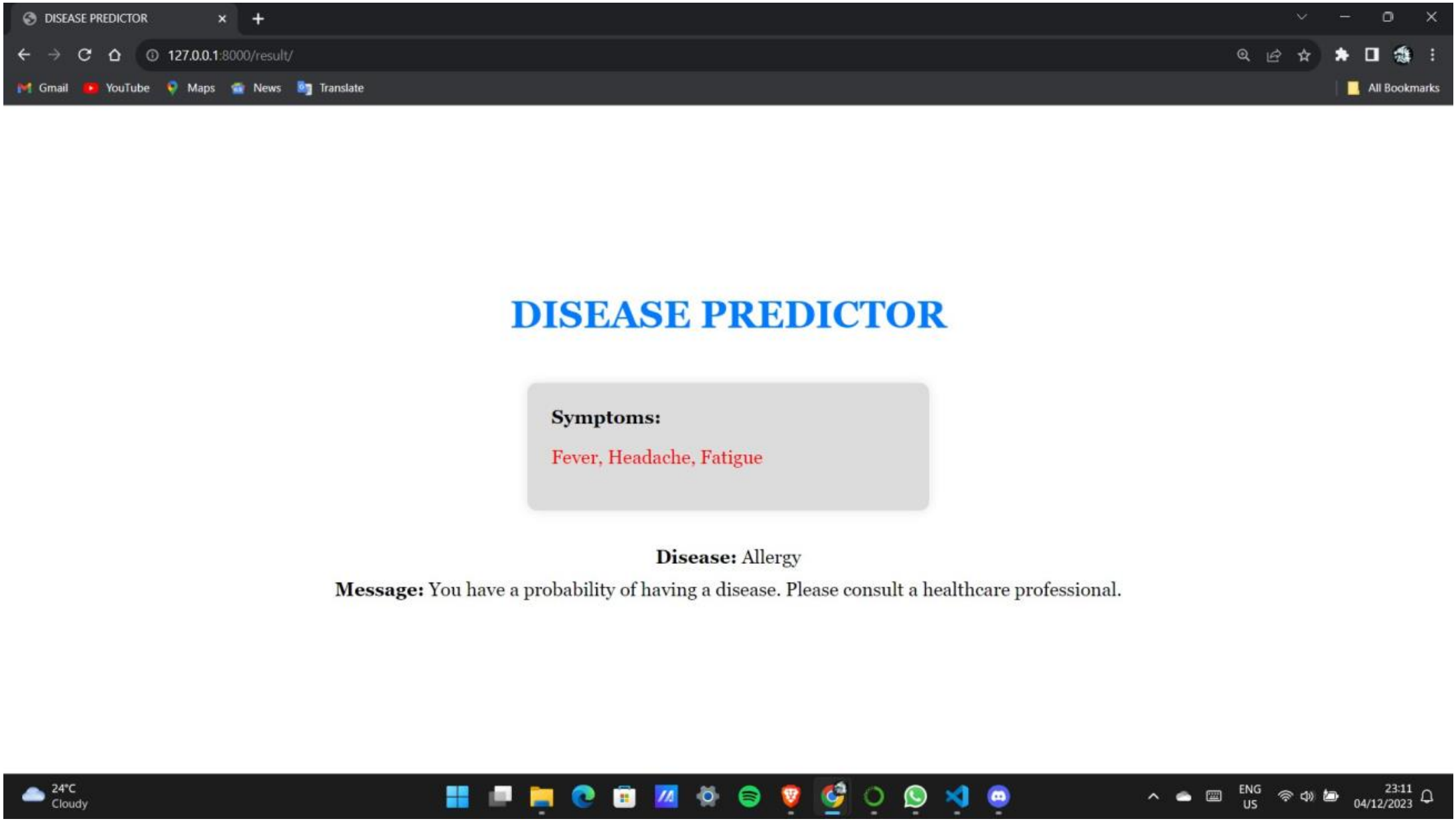
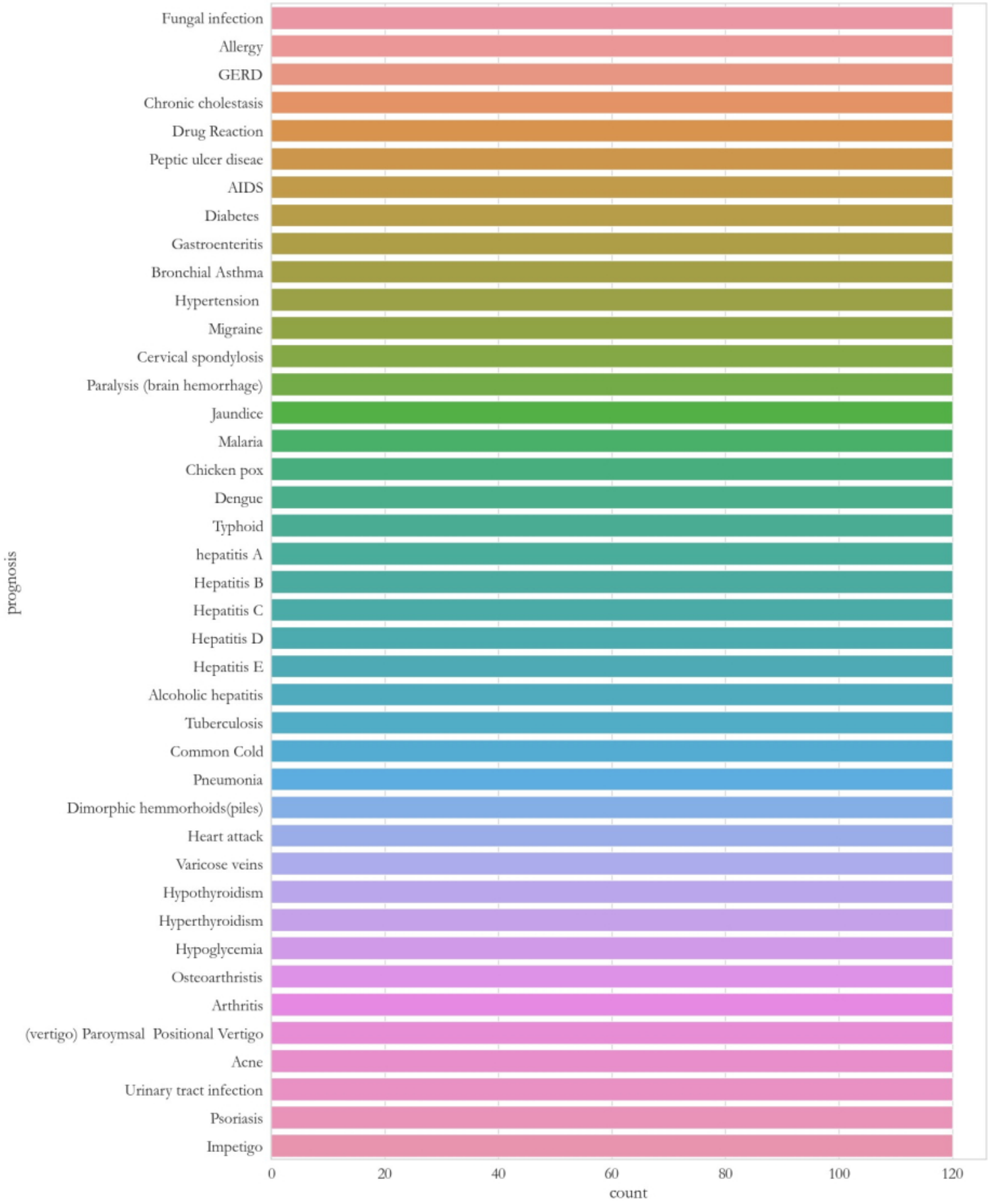
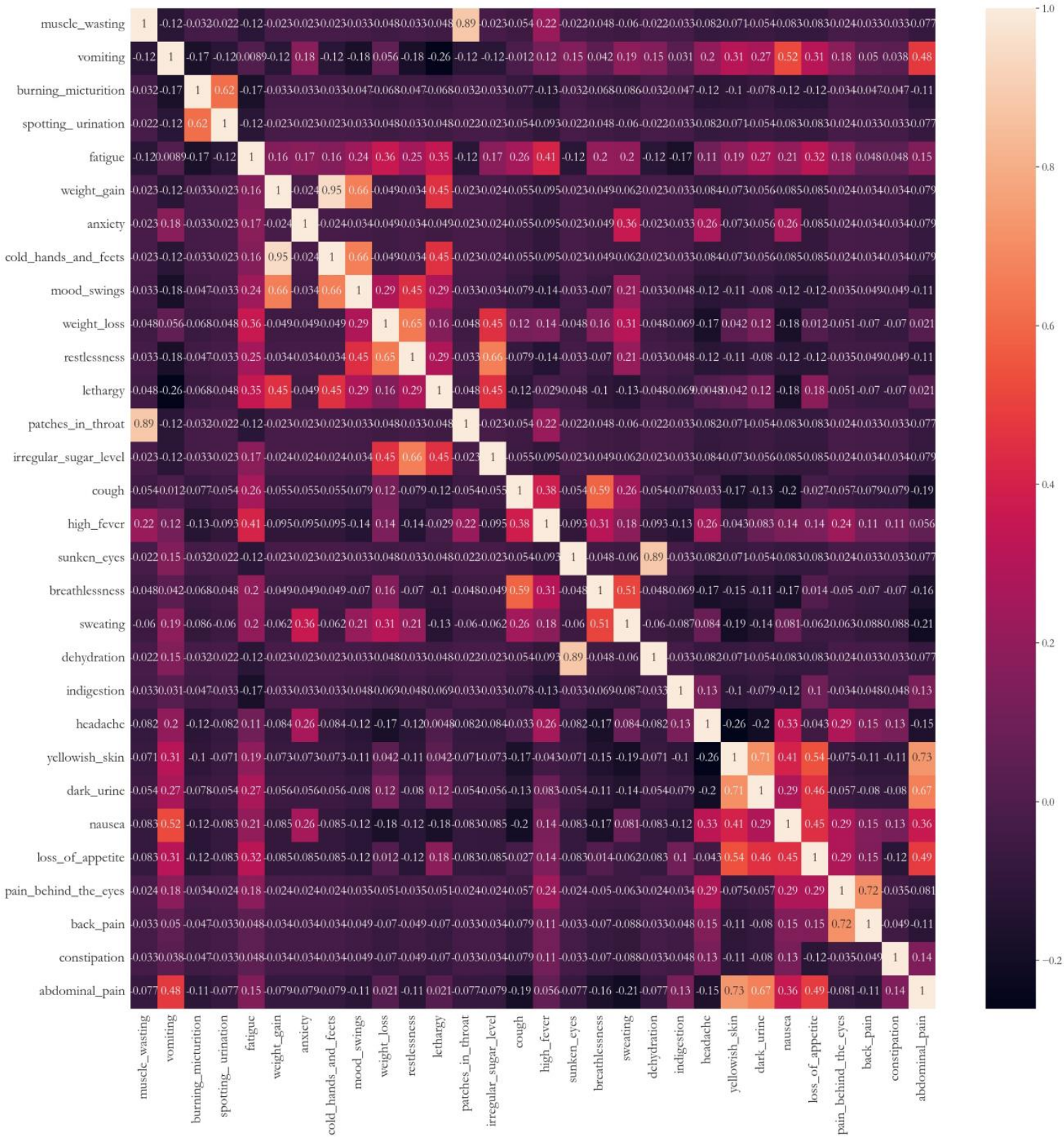

DISEASE PREDICTOR
127.0.0.1:8000
Gmail
YouTube
Maps
News
Translate
All Bookmarks
DISEASE PREDICTOR
Enter Symptoms (comma-separated):
Fever, Headache, Fatigue
Predict
24°C
Cloudy
ENG
US
23:08
04/12/2023


DISEASE PREDICTOR
127.0.0.1:8000/result/
Gmail
YouTube
Maps
News
Translate
All Bookmarks
DISEASE PREDICTOR
Symptoms:
Fever, Headache, Fatigue
Disease: Allergy
Message: You have a probability of having a disease. Please consult a healthcare professional.
24°C
Cloudy
ENG
US
23:11
04/12/2023


Fungal infection
Allergy
GERD
Chronic cholestasis
Drug Reaction
Peptic ulcer diseae
AIDS
Diabetes
Gastroenteritis
Bronchial Asthma
Hypertension
Migraine
Cervical spondylosis
Paralysis (brain hemorrhage)
Jaundice
Malaria
Chicken pox
Dengue
Typhoid
hepatitis A
Hepatitis B
Hepatitis C
Hepatitis D
Hepatitis E
Alcoholic hepatitis
Tuberculosis
Common Cold
Pneumonia
Dimorphic hemmorhoids(piles)
Heart attack
Varicose veins
Hypothyroidism
Hyperthyroidism
Hypoglycemia
Osteoarthristis
Arthritis
(vertigo) Paroymsal Positional Vertigo
Acne
Urinary tract infection
Psoriasis
Impetigo
prognosis
0
20
40
60
80
100
120
count


muscle_wasting
vomiting
burning_micturition
spotting_ urination
fatigue
weight_gain
anxiety
cold_hands_and_feets
mood_swings
weight_loss
restlessness
lethargy
patches_in_throat
irregular_sugar_level
cough
high_fever
sunken_eyes
breathlessness
sweating
dehydration
indigestion
headache
yellowish_skin
dark_urine
nausea
loss_of_appetite
pain_behind_the_eyes
back_pain
constipation
abdominal_pain
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

## Model - Decision Tree Classifier

In [20]:
```
dt_model = DecisionTreeClassifier(random_state = 42)
dt_model.fit(x_train, y_train)

y_pred=dt_model.predict(x_test)

accuracy = metrics.accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 97.61904761904762%
```

Model - Random Forest Classifier

In [21]:
```
rfc_model = RandomForestClassifier(random_state = 42, n_estimators = 100)
rfc_model.fit(x_train, y_train)

rfc_pred=rfc_model.predict(x_test)

accuracy = accuracy_score(y_test, rfc_pred)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 97.61904761904762%
```

## Model - Gradient Boosting Classifier

In [22]:
```
gb_model = GradientBoostingClassifier()
gb_model.fit(x_train, y_train)

gb_pred = gb_model.predict(x_test)

accuracy = accuracy_score(y_test, gb_pred)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 97.61904761904762%
```

In [25]:
```
lr_model = LogisticRegression()
lr_model.fit(x_train, y_train)

lr_pred = lr_model.predict(x_test)

accuracy = accuracy_score(y_test, lr_pred)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 100.0%
```

## Model - k-Nearest Neighbors (KNN)

In [23]:
```
knn_model = KNeighborsClassifier()
knn_model.fit(x_train, y_train)

knn_pred = knn_model.predict(x_test)

accuracy = accuracy_score(y_test, knn_pred)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 100.0%
```

## Model - Naive Bayes

In [19]:
```
model = GaussianNB()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = metrics.accuracy_score(y_test, y_test)
print(f"Accuracy: {accuracy*100}%")
```

```
Accuracy: 100.0%
```